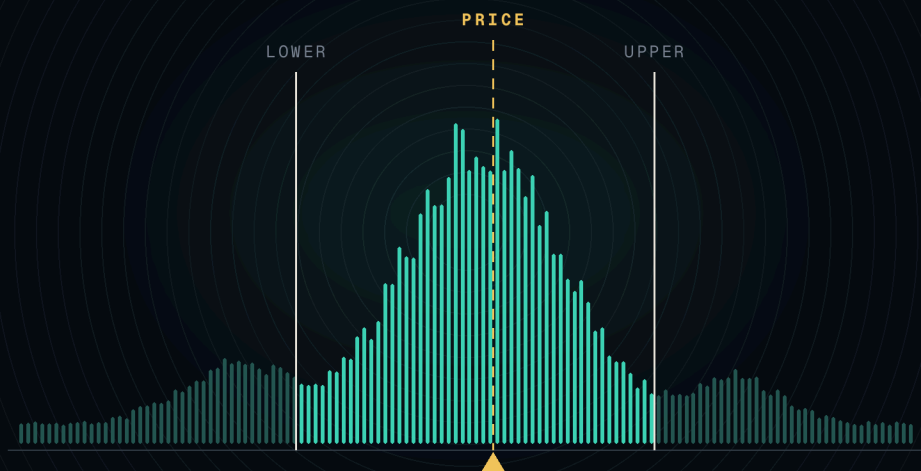


A FIELD MANUAL FOR CONCENTRATED LIQUIDITY ON SOLANA

Liquidity on *Autopilot*

*Strategies for Orca Whirlpools with Aeternum,
an open-source AI liquidity agent*



THE MODEL CHOOSES ENTRIES · ARITHMETIC CHOOSES EXITS

AETERNUM

FIRST EDITION · 2026

A FIELD MANUAL FOR CONCENTRATED LIQUIDITY ON SOLANA

Liquidity on *Autopilot*

Strategies for Orca Whirlpools with Aeternum, an open-source AI liquidity agent

A E T E R N U M

FIRST EDITION • 2026

LIQUIDITY ON AUTOPILOT

Strategies for Orca Whirlpools with Aeternum. First edition, October 2026.

SOFTWARE

Aeternum is free and open-source under the MIT licence: github.com/pecundangcrypto/aeternum. The project is inspired by Meridian, an earlier open-source liquidity agent, and shares its concept, not its code. Orca, Whirlpools, Solana and Jupiter are trademarks of their respective owners; this book is independent and not endorsed by any of them.

NOT FINANCIAL ADVICE

This book is education about software and market mechanics. Providing liquidity can lose money, quickly and in full. Nothing here is a recommendation to buy, sell or deposit any asset, and no figure in it is a promise of return. Every number drawn from a live deployment is reported as it happened, small sample and all. Run everything in paper mode first, deploy only what you can afford to lose, and make your own decisions.

VERSIONS

Written against Aeternum as of October 2026, Orca Whirlpools SDK v8 and the Orca v2 public API. Defaults change; when this book and the code disagree, the code in `src/config.js` is the authority.

COPYRIGHT

© 2026 pecundangcrypto. All rights reserved for the text of this book. Code excerpts are from Aeternum and remain under its MIT licence.

Contents

–	Before you begin	5
PART I • MECHANICS		
01	How a Whirlpool position really behaves	9
02	The cost of getting in and out	14
03	Inside Aeternum	18
PART II • STRATEGY		
04	Choosing pools	23
05	Range geometry: width and skew	28
06	The strategy playbook	32
07	Exits are arithmetic	40
08	Sizing and risk	44
09	Learning: lessons, evolution, hivemind	47
PART III • OPERATION		
10	From paper to live	51
11	Measuring results honestly	55
12	Field notes from a live deployment	58
13	Checklists and troubleshooting	62
REFERENCE		
A	Settings reference	65
B	Glossary	68

Before you begin

What this book is, what it is not, and the one idea that runs through all of it.

CONCENTRATED LIQUIDITY IS ONE OF THE MOST ELEGANT IDEAS IN DECENTRALISED FINANCE and one of the easiest to lose money with. A Whirlpool position on Orca can earn fees at many times the rate of an ordinary pool, because your capital is packed into a narrow band of prices instead of being smeared from zero to infinity. The same narrowness is what makes it unforgiving. The moment price walks out of your band you stop earning, and you are left holding whichever asset just lost value.

Doing this well by hand means watching charts at three in the morning, re-reading pool statistics every hour, and closing positions at exactly the moment you least want to. Most people who try it discover that the hard part is not the theory. It is the discipline.

Aeternum is an attempt to automate that discipline. It is an open-source agent that screens Orca pools, opens positions with ranges sized to each pair's volatility, and manages them around the clock: trailing take-profit, stop loss, out-of-range exits, fee harvesting, and a Telegram bot so you can see and override everything from your phone. It is free. You can read every line of it.

This book is the manual I wish had existed when building it. It explains the mechanics underneath every decision, the strategies the agent can run and when each one fits, the settings that matter and the ones that do

not, and, in Chapter 12, what actually happened when it went live with real money, including the mistakes that cost money.

The model chooses entries.
Arithmetic chooses exits.

THE DESIGN RULE BEHIND EVERYTHING IN THIS BOOK

That rule is the spine of the book. A language model is good at weighing messy evidence (is this token legitimate, is this pool's volume organic, is this the right shape of range for this pair) and bad at being fast, consistent and boring. A stop loss has to be all three. So in Aeternum the model may propose and open positions, but nothing that closes a position runs through it. Exits are a short list of rules evaluated by plain code every twenty seconds. You will see this idea return in every chapter, because most of the money in liquidity provision is lost not by choosing the wrong pool but by holding the wrong position for too long.

Who this book is for

You do not need to be a developer. You do need to be comfortable running a few commands in a terminal, editing a configuration file, and keeping a small server or always-on computer running. If you have ever provided liquidity on any DEX, you will move quickly. If you have not, Chapter 1 starts from zero.

What this book will not do

It will not promise you a return. There are no screenshots of improbable profit here, because a strategy that is honest about its costs does not produce them. What this book offers instead is understanding: why a range

behaves the way it does, what each setting trades away, and how to judge whether the agent is actually working, so that any money you put in is put in with your eyes open.

READ THIS ONCE, SERIOUSLY

Everything in this book can be tested in **paper mode**, where the agent runs against live markets on a simulated balance with no private key on the machine at all. Run it that way for several days before you fund anything.

When you do go live, start with an amount whose total loss would be annoying rather than painful.

How to read it

Part I is mechanics: how positions behave, what they cost, and how the agent is put together. Part II is strategy: choosing pools, shaping ranges, six complete playbooks with configuration, exits, sizing and learning. Part III is operation: setting it up, measuring it honestly, and the field notes. If you are impatient, read Chapters 1, 2 and 6, then come back for the rest.

PART I

Mechanics

*Before any strategy: what a position is, what it costs, and
what the machine around it does.*

How a Whirlpool position really behaves

Uniform liquidity between two prices, and the three consequences that decide every outcome.

AN ORDINARY CONSTANT-PRODUCT POOL SPREADS YOUR DEPOSIT ACROSS EVERY POSSIBLE price, from almost zero to almost infinity. Nearly all of that capital sits at prices the market will never visit, so nearly all of it earns nothing. Orca's Whirlpools, like Uniswap v3 before them, let you choose a band instead. You say: I will provide liquidity only while the price is between *here* and *there*. Inside that band your capital is dense and earns a large share of every swap. Outside it, it does nothing at all.

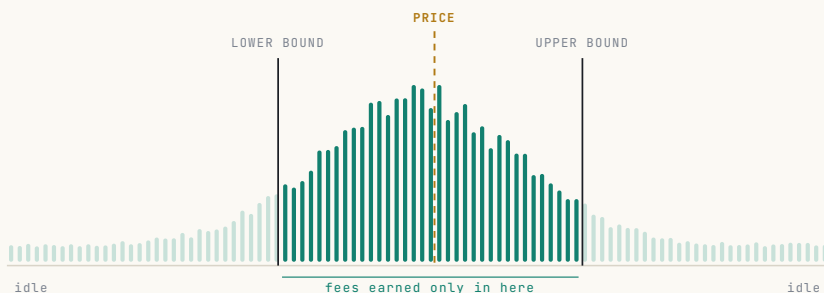


FIG. 1.1 A position is uniform liquidity between two bounds. Inside, it earns a share of every swap; outside, nothing.

Ticks and tick spacing

Prices on a Whirlpool live on a logarithmic grid of integers called **ticks**. Each tick is a 0.01% step in price. A pool does not let you start or end a position on any tick, only on multiples of its **tick spacing**: commonly 1, 2, 4, 8, 16, 64, 96, 128 or 256, with larger spacings usually paired with higher fee tiers. A pool with spacing 64 can only place range edges roughly 0.64% apart.

This matters more than it sounds. Ask for a range 3% wide on a pool with spacing 256, where one step is about 2.6%, and there is barely one valid band to choose from. Ask for one narrower than a single spacing and both edges snap to the same tick, which the program rejects. Aeternum's `buildRange` handles this for you: it snaps both edges to the grid, rounds them *outward* so the range never shrinks below what you asked for, and widens by one spacing if snapping would have produced a zero-width range.

Which way is the price?

A Whirlpool always quotes its price as *token B per token A*, and which token is A is decided by comparing the two mint addresses byte by byte. That is essentially random. On one pool the volatile asset is token A and the price reads "USDC per BONK"; on the next it is token B and the price is upside down.

Aeternum never assumes. For every pool it works out which side is the **base** (the asset you are farming, the volatile one) and which is the **quote** (what you measure value in: SOL, USDC or USDT). Every range in this book, and every range you configure, is expressed as the base price in quote terms, then mirrored into pool orientation behind the scenes. When you read "skew 0.9, range below price", it means below the price of the base asset, whatever the pool's internal orientation happens to be.

Consequence one: fees only accrue in range

Each swap pays a fee, and that fee is divided among the liquidity that is *active at the current price*. A position whose range does not contain the price receives nothing. There is no partial credit for being close. This single fact explains why time in range, which Aeternum records for every position as `rangeEfficiency`, is the most useful number for judging whether a range was the right size.

Consequence two: narrow earns more, and leaves sooner

The narrower the range, the more liquidity the same dollars buy, and the larger your share of each fee. Compared with spreading the same deposit across the full price curve, a symmetric range 30% wide is about fourteen times as capital-efficient; 12% wide is about thirty-four times; 5% wide is about eighty times. Those multipliers are real, and they are why concentrated liquidity is attractive at all.

The catch is that a 5% range on an asset that moves 8% in an afternoon will spend most of that afternoon out of range, earning nothing. Width is a direct trade between **fee density** and **dwelt time**. There is no universally correct width, only a correct width *for a given pair's volatility*, which is why Aeternum sizes ranges from each pool's own behaviour (Chapter 5).

SYMMETRIC WIDTH	CAPITAL EFFICIENCY VS FULL RANGE	TYPICAL CHARACTER
5%	≈ 80×	Very dense; needs a calm pair or constant attention
12%	≈ 34×	The workhorse width for liquid mid-caps
30%	≈ 14×	Forgiving; earns less per dollar, rarely leaves range

Computed at entry, for a range centred on the price.

Consequence three: divergence is the real opponent

A liquidity position is constantly rebalancing against you. As price falls, swaps sell you the falling asset; as it rises, they buy it back from you. By the time price exits the bottom of your range you hold *only* the base asset, bought all the way down. Exit through the top and you hold only quote, having sold the base all the way up. Compared with simply holding the two assets, you always end up worse off. That shortfall is called divergence loss, or impermanent loss, though nothing about it is impermanent once you close.

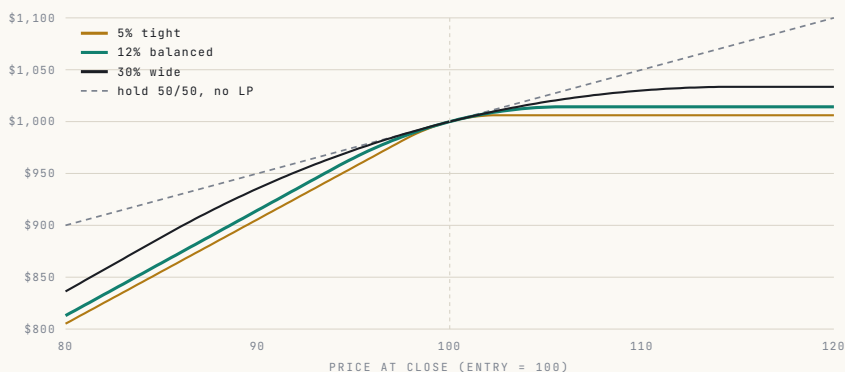


FIG. 1.2 Value of a \$1,000 symmetric position at close, before fees, for three widths. Every curve sits below holding; fees have to make up the gap. Narrower ranges diverge harder.

The figure makes the trade visible. Take a \$1,000 deposit at a price of 100. In a 12% symmetric range (94 to 106), a move down to 94 leaves the position worth about \$955, roughly 1.6% behind simply holding. A move down to 88, outside the range, leaves it worth \$894: still converting at worse and worse prices until it stopped at 94, then simply riding the base asset lower. Above the range the position is capped. At 120 it is still worth only about \$1,014, because it sold every unit of base on the way up.

A range that collects 3% in fees while the pair moves 15% is a loss. Fee APR only means something relative to the volatility that has to pay for it.

Every strategy in this book is a different answer to one question: *how do I collect more in fees than I give away to divergence?* You can make the range wide enough that divergence is gentle. You can choose pools whose fee flow is enormous relative to how far they move. You can make the range one-sided so that divergence works like a limit order you were happy to have filled. And you can leave quickly when the trade stops working. Aeternum does all four; the art is in the settings.

A position's life, end to end

- 1. Funding.** The wallet holds SOL. Whatever part of the deposit is not SOL is bought, usually through Jupiter.
- 2. Open.** A position NFT is minted, the range's tick accounts are initialised if needed, and liquidity is deposited in the exact ratio the range demands at the current price.
- 3. Earning.** While in range, fees accumulate inside the pool against the position's checkpoint. They are not paid out automatically.
- 4. Harvest (optional).** Collecting fees moves them to your wallet without touching the liquidity.
- 5. Close.** Liquidity is withdrawn, remaining fees are collected, and the position NFT is burned, refunding its rent. Aeternum then sells any non-SOL proceeds back to SOL by default, so capital does not drift one exit at a time into tokens you never meant to hold.

Each of those steps costs something. The next chapter adds it up.

The cost of getting in and out

Swaps, spreads and rent: the line item that silently decides whether a small position can ever be profitable.

MOST DISCUSSIONS OF LIQUIDITY PROVISION COMPARE FEE APR AGAINST IMPERMANENT loss and stop there. On a small wallet that misses the single largest cost of all, the cost of entering and leaving the position. It does not show up on Orca's position page. It does show up in your wallet.

Where the money goes

COST	WHEN	SIZE
Funding swap	Open	Swap fee plus spread on every non-SOL part of the deposit
Exit swap	Close	The same again, selling non-SOL proceeds back to SOL
Position rent	Open, refunded at close	About 0.0068 SOL; comes back when the NFT is burned
Tick array growth	Open, sometimes	About 0.0016 SOL when a range edge lands on a fresh tick; usually not refunded
Network and priority fees	Every transaction	Small, but they add up over many opens and closes

Aeternum models swap cost as a fee plus a spread on each leg, and adds a fixed cost of about 0.0016 SOL per position for tick growth and network fees. Those are deliberately conservative round numbers rather than live quotes. The fixed part is what punishes small positions:

POSITION SIZE	FIXED COST SHARE
0.25 SOL	≈ 0.64%
1 SOL	≈ 0.16%
5 SOL	≈ 0.03%

The shape of the pool decides the bill

Here is the most practically useful idea in this chapter: **funding cost is not a property of the pool alone, but of the pool together with the range**. A position below the current price is held entirely in the quote asset. A position above it is held entirely in the base asset. Only a range that straddles the price needs both. Combine that with the fact that your wallet holds SOL, and three shapes fall out:

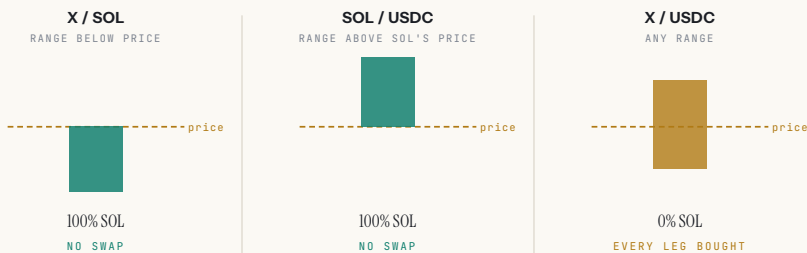


FIG. 2.1 What share of the deposit is SOL, and therefore free to fund, depends on the pool and on where the range sits relative to price.

- **X/SOL, range below price** (skew near 1). SOL is the quote, and the whole range sits below the base price, so the deposit is 100% SOL. No swap on the way in. If price never falls into the range, there is no swap on the way out either.
- **SOL/USDC-style, range above SOL's price** (skew near 0). Here SOL is the *base*, and a range above its price is held entirely in SOL. Again, no swap in.
- **X/USDC, any range**. There is no SOL in the pool at all, so every leg is bought on the way in and sold on the way out. This is the most expen-

sive shape there is.

MEASURED, NOT ESTIMATED

Aeternum's first live position was DOGE/USDC, both legs bought from SOL. Measured from the wallet, the round trip came to roughly **1.7%**, which is about the entire edge of a typical position. The pool was fine. The shape was expensive. That one measurement is why the screener now ranks pools by what they cost to enter.

How Aeternum accounts for it

For every candidate pool the screener computes a **funding block**: the estimated round-trip cost at the configured geometry, and, when SOL is in the pool, the cost at the **zeroSwap** skew that needs no swap at all. It then ranks candidates by

$$\text{rankScore} = \text{yieldScore} - 10 \times (\text{cheapest round-trip cost, in \%})$$

so one percentage point of round-trip cost costs ten points of score. A pool scoring 85 that needs every leg bought (about 1.9% on a small position) ranks below a pool scoring 76 that can be entered for free. The model sees both numbers and is told to prefer the cheaper entry unless the expensive pool's yield clearly pays for the difference.

For live positions, the actual cost is measured, not modelled. Aeternum snapshots the wallet before and after funding and opening, records SOL spent, rent that will come back and any leftover tokens, and uses that to compute **net PnL**: what the wallet would be up or down if the position were closed now and everything sold back to SOL. Chapter 11 explains when to look at net PnL and when to look at position PnL.

The creator fee

Aeternum ships with an optional, disclosed creator fee: a 0.5% Jupiter referral fee on the swaps the agent makes, which funds the project. It is announced at startup, in the README, in the setup wizard and on the dashboard. It never touches your liquidity or your fees, only swaps. It also raises the cost of every non-SOL leg, which is one more reason to prefer zero-swap shapes. To turn it off, set one line in `.env`:

```
AETERNUM_REFERRAL_FEE_BPS=0
```

With the fee off, Aeternum assumes about 0.2% per swap (fee plus spread); with it on, about 0.6%. The cost model follows whichever you choose.

The cheapest position
is the one that needs *no swap*.

SMALL WALLETS: READ THIS TWICE

Inside Aeternum

Two cycles, one watcher, a ledger and a set of gates. How the pieces fit, and why they are shaped this way.

————

AETERNUM IS A SINGLE NODE.JS PROCESS. IT NEEDS A SOLANA RPC ENDPOINT, ANY OpenAI-compatible model endpoint with tool calling, and optionally a Telegram bot. Everything it knows lives in plain JSON files under `data/`. There is no database and no build step. Knowing how it is put together will make every later chapter easier.

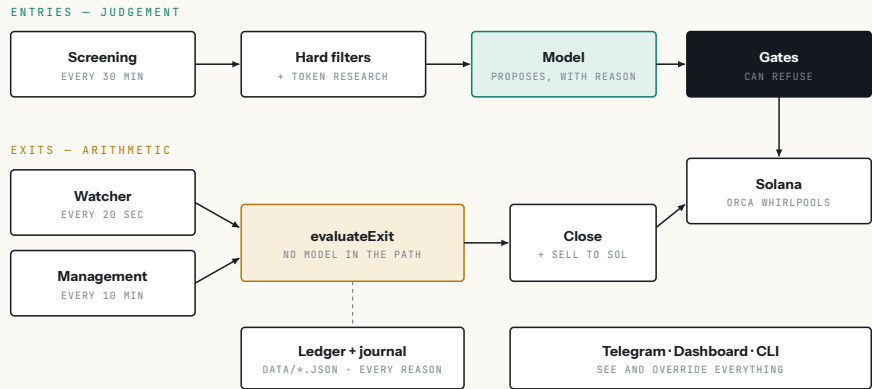


FIG. 3.1 Entries pass through filters, the model and the gates. Exits are decided by plain code on every watcher tick and management cycle.

The screening cycle (every 30 minutes)

Screening pulls the Orca pool list, applies a set of **hard filters** with no model involved (TVL, volume, fee APR, turnover, tick spacing, 24-hour price move, warnings, dangerous token extensions), researches the sur-

vivors' tokens (market cap, holders, concentration, age, Orca's own risk score), computes a Yield Score and funding cost for each, and hands the best handful to the model. The model reads them alongside your lessons, your recent performance and the current thresholds, and may open a position, with a stated reason, through a tool call.

The management cycle (every 10 minutes)

Management values every open position, runs the exit rules, harvests fees when they cross a threshold, reconciles the ledger against what is actually on-chain, and lets the model add notes or lessons. The model can also ask to close a position early, but the rules close positions whether or not the model agrees.

The watcher (every 20 seconds)

Between cycles, a lightweight watcher re-values positions and runs the same exit engine. This is what makes the trailing take-profit useful: a ten-minute loop would give back far more than the trailing distance before it noticed. The watcher can run on a second, cheaper RPC so it never eats your main endpoint's rate limit.

The gates

Every action the model takes goes through code that can refuse it. An open is refused if the position limit is reached, the pool is already held, the base token is already held elsewhere, the pool is blocked or cooling down, the pool's score has dropped since screening, the wallet is below its minimum, or what is left after the gas reserve is smaller than the minimum position. Sizes above your maximum are capped, and range widths outside the allowed band are clamped to it. A refusal is returned to the model as an error with the reason, so it can adjust instead of retrying blindly.

WHY GATES, NOT PROMPTS

"Never hold more than three positions" written in a prompt is a suggestion. The same rule in the executor is a guarantee. Aeternum puts every risk limit in code, and treats the prompt as a place to explain the market, not to enforce safety.

The ledger and the journal

On-chain accounts say what a position *is*. They say nothing about why it was opened, what it cost, its peak PnL or how long it has been out of range. The ledger in `data/positions.json` holds that half, and the exit engine runs on it. Every mutating action also takes a written `reason` that lands in `data/journal.json`, so you can always answer "why did it do that?" with the agent's own words at the time.

Paper and live books

With `DRY_RUN=true` and no wallet key configured, Aeternum runs on a synthetic SOL balance and cannot sign anything, because there is no key on the machine. Paper positions are valued against live pool state using the same maths the on-chain program uses; only fee income is estimated, and every paper snapshot is flagged as such. Paper and live positions are kept in separate books, and every close, harvest and sweep refuses to touch a position from the other book, so a test run can never act on real money.

The interfaces

INTERFACE	BEST FOR
Telegram	Daily use: status, positions with range bars, close by index with confirmation, live setting tweaks, open/close alerts
Dashboard	A read-only status page (port 8788) for a glance at strategy return, positions and history
Terminal REPL	Running a cycle now, inspecting config, chatting with the agent
CLI	One-shot JSON commands; write commands are dry runs unless you pass <code>--live</code>

PART II

Strategy

Which pools, what shape of range, how big, and when to leave. Six complete playbooks and the reasoning behind each setting.

Choosing pools

*Hard filters reject the obviously unsuitable. A score ranks what survives.
The model judges the rest.*

ORCA LISTS THOUSANDS OF WHIRLPOLS. MOST ARE DEAD, MANY ARE DANGEROUS, and a few are genuinely good places to provide liquidity on any given day. Aeternum finds those few in three stages, and the order matters: cheap, deterministic checks first; expensive research only on the survivors; judgement last, and only over candidates that have already passed everything a rule can check.

Stage one: hard filters

These run on the whole pool list with no network calls per pool and no model. A pool that fails any of them is rejected with a stated reason, and the reasons are kept. When a screening cycle finds nothing, the rejection list tells you why, which is the usual answer to "is the screener broken?"

FILTER	DEFAULT	WHAT IT PROTECTS AGAINST
<code>minTvlUsd</code>	\$25k	Pools too thin to absorb your position without you becoming the market
<code>maxTvlUsd</code>	\$3M	Pools so deep your share of fees is negligible
<code>minVolume24hUsd</code>	\$50k	No volume, no fees
<code>minFeeApr</code>	40%	Pools that cannot outrun divergence; fees ÷ TVL, annualised
<code>minVolumeTvlRatio</code>	1.5×	Idle capital. Turnover is what actually pays an LP
<code>maxPriceDelta24h</code>	60%	Pairs moving so fast any concentrated range is left behind
<code>min/maxTickSpacing</code>	1–256	Grids too coarse for the widths you want
<code>rejectWarningPools</code>	on	Pools Orca itself flags
<code>blockedTokenExtensions</code>	six	Token-2022 features hostile to LPs (below)
<code>minYieldScore</code>	45	Everything mediocre, in one number

TOKEN-2022 EXTENSIONS

Solana's newer token standard lets a mint carry extensions, and several of them are incompatible with leaving capital in a pool. A **transfer hook** runs arbitrary third-party code on every transfer, including every swap through your pool. A **permanent delegate** holds standing authority to move tokens out of any account, in principle including the pool vault. A **pausable** mint can freeze the asset while your capital is locked in a range. A **transfer fee** silently breaks the constant-product maths the position is priced on. Aeternum blocks these by default, along with default-frozen and non-transferable mints. Leave this list alone unless you know exactly why you are changing it.

Stage two: token research

Survivors have their base token researched through Jupiter and Orca. Each check is a common way a token kills a liquidity position:

CHECK	DEFAULT	WHY
Orca token risk	≤ 4 of 10	Orca's own rating; the cheapest strong signal available
Market cap	\$250k – \$500M	Too small is a rug risk; too large rarely needs your liquidity
Holders	≥ 400	Thin holder bases are easy to dump
Top-10 concentration	$\leq 60\%$	A few wallets can drain the price into your range
Token age	$\geq 24\text{h}$	Most launch-day catastrophes happen on launch day

A token with no data at all is rejected. For anything other than a known quote asset, being unverifiable is itself a red flag.

Stage three: the Yield Score

Pools differ by orders of magnitude in size and yield, so Aeternum compresses each into a single 0–100 **Yield Score** made of four parts. Each part saturates on a logarithmic scale, so that no single dimension can dominate and the ranking still distinguishes between pools at the high end where it matters.

COMPONENT	POINTS	FULL MARKS AT
Fee yield (annualised fee APR)	40	1,200% APR
Turnover (volume $\div$ TVL)	25	20 $\times$ per day
Depth (TVL)	20	\$2M
Stability (inverse 24h move)	15	Under 5%; zero at 50%

The Yield Score is explicitly *not* a prediction of profit. It is a ranking device that pushes the unsuitable to the bottom so the model spends its reasoning on plausible candidates. The final rank also subtracts funding cost (Chapter 2), so a cheap-to-enter pool can outrank a higher-scoring one.

What the model adds

By the time a pool reaches the model, every rule-checkable question has been answered. What remains is judgement: is this volume organic or one wallet washing trades? Is a 40% move today the start of a trend or the end of a spike? Does a lesson from last week apply here? Does the expensive pool's yield really justify the swap cost? The model answers in writing, and that reasoning is stored with the position.

Signals: nominating pools

You can push a pool or mint to the front of the queue from anywhere that can run a command:

```
aeternum signal add <pool-or-mint> --source discord --note "called in #alpha"
aeternum signal list
```

Signals are nominations, not instructions. A signalled pool still has to clear every filter. They de-duplicate over ten minutes and expire after an hour. Set `signalMode` to `only` to screen nothing but your signals.

Three starting profiles

The setup wizard offers three coherent sets of thresholds. They are good starting points precisely because the values are consistent with each other: a conservative screener paired with an aggressive exit policy makes no sense.

SETTING	CONSERVATIVE	BALANCED	AGGRESSIVE
Max positions	2	3	4
Position size (of deployable)	20%	35%	50%
TVL range	\$150k–20M	\$25k–3M	\$15k–1M
Min fee APR	25%	40%	100%
Min turnover	1×	1.5×	3×
Max 24h move	25%	60%	100%
Min market cap	\$5M	\$250k	\$100k
Min holders	5,000	400	200
Min token age	14 days	24h	6h
Min Yield Score	40	45	55
Range preset	wide	ladder_bid	tight
Take profit / stop loss	6 / -8%	8 / -12%	12 / -18%
Trailing trigger / drop	3 / 1%	4 / 1.5%	5 / 2%
Out-of-range wait	40 min	25 min	12 min
Max hold	168h	72h	24h

THE SINGLE BIGGEST LEVER

If you change only one screening setting, make it `maxPriceDelta24h`. It is the main volatility gate and the largest single control on divergence loss. Halving it removes most of the pools that produce stop-loss exits, and most of the excitement with them.

Range geometry: width and skew

Two numbers describe every range Aeternum can open. Learn what each one trades away.

BECAUSE A WHIRLPOOL POSITION IS NOTHING MORE THAN UNIFORM LIQUIDITY BETWEEN two prices, every "strategy" at the position level is purely geometric. Aeternum describes it with two numbers:

- **width**, the total span of the range as a percentage of the entry price;
- **skew**, the share of that width placed *below* the base asset's price, from 0 to 1.

SKREW 1.0
all below · bid ladder · enters in quote

SKREW 0.92
ladder_bid default · mostly below

SKREW 0.5
centred · both assets

SKREW 0.0
all above · ask ladder · enters in base

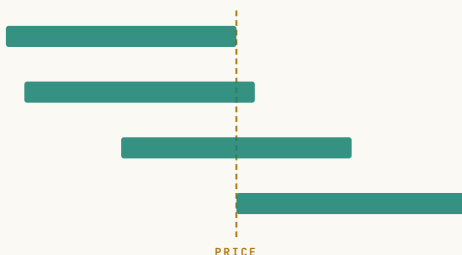


FIG. 5.1 Same width, four skews. Skew is the share of the range placed below the base asset's price.

Reading skew

Skew 0.5 centres the range on the price. You enter with both assets and earn in both directions. This is the classic liquidity position.

Skew 1.0 puts the entire range below the price. You enter entirely in the quote asset, earn nothing until the price dips into the range, and then buy the base asset gradually as it falls, collecting fees as you go. This behaves like a ladder of resting buy orders that pays you to wait. If the quote is SOL, it needs no funding swap at all.

Skew 0.0 puts the range above the price. You enter entirely in the base asset and sell it gradually into strength. This is a ladder of take-profit orders that also earns fees.

Everything in between is a blend. A skew of 0.92 places 92% of the width below the price and a sliver above, so the position starts earning immediately but is still mostly a bid.

The five presets

PRESET	WIDTH	SKEW	BEHAVIOUR
ladder_bid	12%	0.92	Mostly below price; accumulates base on dips. The default
balanced	12%	0.50	Symmetric around price
tight	5%	0.50	Maximum fee density; leaves range fast
wide	30%	0.50	Low density, long dwell time, gentle divergence
exit_ask	12%	0.08	Mostly above price; distributes base into strength

Set the default with `rangePreset` . Override either number globally with `rangeWidthPct` and `rangeSkew` . The model may also choose width and skew per position when a pool's volatility argues for it, and must say why.

Adaptive width

A fixed width is wrong for almost every pool, because pools differ enormously in how much they move. With `adaptiveRangeWidth` on (the default), Aeternum replaces the preset's width with an estimate from the pool's own behaviour. It takes two independent estimates and uses the wider:

```
net-move width = |24h price change| × 100 × adaptiveRangeWidthFactor (0.75)
turnover width = √(volume ÷ TVL) × adaptiveTurnoverFactor (3)
width           = max(both), clamped to minRangeWidthPct ...
maxRangeWidthPct (6 ... 60)
```

The turnover term exists because the 24-hour price change is a *net* number. A pair that swings ten percent each way during the day and closes flat reports roughly zero, which on its own would size a range at the floor and see it abandoned within minutes. Turnover does not cancel out like that. A pool turning over its TVL many times a day is being traded hard, and the range needs room. The square root is there because turnover grows much faster than the price range it implies.

24H MOVE	TURNOVER	NET-MOVE WIDTH	TURNOVER WIDTH	RESULT
2%	1×	1.5%	3%	6% (floor)
4%	9×	3%	9%	9%
18%	4×	13.5%	6%	13.5%
40%	16×	30%	12%	30%

The skew still comes from your preset or override. Adaptive mode only sizes the width, but it sizes it *instead of* the preset and instead of `rangeWidthPct`. If you want a fixed width, turn `adaptiveRangeWidth` off. Note too that every width, adaptive or not, is clamped to `minRangeWidthPct`, which defaults to 6%: the `tight` preset's 5% only takes effect if you lower that floor.

Tuning the width factor

The width factor is the setting you are most likely to tune after a week of results, and the agent's own evolution logic does it too (Chapter 9). The signal is in two numbers you can read from `aeternum performance`:

- If **out-of-range** is your most common exit reason, ranges are too narrow for the pools being picked. Raise the factor.
- If average **time in range** is above 90% and out-of-range exits are rare, ranges are wider than necessary and you are giving up fee density. Lower it a little.

One-sided ranges and the price edge

A subtle detail: when skew is exactly 1 or exactly 0, the range edge that touches the current price is rounded *away* from it during tick snapping. Rounding outward as usual would push that edge across the price by up to one tick spacing, turning a "no swap needed" entry into one that needs a few percent of the other asset bought, swap fee and all. If you want a truly zero-swap entry, use a skew of exactly 1 (SOL as quote) or 0 (SOL as base), not 0.95, and read the warning below first.

FREE IS NOT THE SAME AS CHEAP

A skew of exactly 1 on an X/SOL pool needs no swap at all, but the range then sits entirely below the price, so the position is **out of range from the moment it opens**. It earns nothing until price dips into it, and if no dip comes within `outOfRangeWaitMinutes` the exit engine closes it, at the cost of network fees and any tick-array rent. The default 0.92 is in range from the start, and the base slice it buys is only about 8% of the deposit, roughly 0.03% to 0.1% of round-trip cost depending on the creator fee. For most wallets 0.92 to 0.95 is the better trade. Use exactly 1 only when you deliberately want a resting bid and have lengthened the out-of-range wait to match.

The strategy playbook

Six complete ways to run Aeternum, each with its idea, the market it suits, how it fails, and the configuration that expresses it.

A STRATEGY, IN AETERNUM, IS A COHERENT COMBINATION OF FOUR DECISIONS: *WHICH* pools may be entered, *what shape* of range to open in them, *how much* to put in, and *when to leave*. Change one without the others and you usually get the worst of both. The playbooks below are complete combinations. Each lists only the settings it changes from the defaults; everything else stays as shipped. The three profiles from Chapter 4 are applied by the setup wizard, which writes their values into the file; the profile name itself is only a label.

Settings go in `user-config.json`, are picked up within a few minutes without a restart, and can also be changed live from Telegram's `/config` or with `aeternum config set <key> <value>`. A few structural settings, such as `minRangeWidthPct` and `maxRangeWidthPct`, are read at startup only; restart the agent after changing those. In the snippets, `SOL` and `USDC` stand for the full mint addresses:

```
SOL    So111111111111111111111111111111111111111111111111112
USDC   EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v
```

NONE OF THESE IS A RECOMMENDATION

These are shapes of strategy, described honestly, not systems that make money. Every one of them can lose. Run any of them in paper mode for several days before trusting it with funds, and compare the result with simply holding SOL over the same period.

1• The Core Bid

RISK	POOLS	RANGE	HOLD
Medium	SOL & USDC	Adaptive • 0.92	Hours to 3 days

The idea. This is Aeternum as shipped. Ranges sit almost entirely below the price, sized to each pool's volatility. The position starts in range and earning, with a small slice above the price. If the price dips it accumulates the base asset at progressively better prices, collecting fees while it does. If the price rises, the thin upper slice is quickly passed and the position is all quote again, waiting. The trailing take-profit locks in gains once they are worth protecting.

It suits liquid mid-cap pairs that chop sideways with an upward bias, the most common state of a healthy Solana token on a normal day.

It fails in a sustained downtrend. The position buys all the way through its range and is left holding the base asset below it. The out-of-range timer and stop loss limit the damage, but they do not prevent it.

There is nothing to change. For reference, the settings that define it are:

```
{
  "rangePreset": "ladder_bid",
  "adaptiveRangeWidth": true
}
```

Watch the exit mix. A healthy Core Bid closes mostly on trailing take-profit and maximum hold, with out-of-range exits a minority. If out-of-range dominates, widen with `adaptiveRangeWidthFactor`.

2· The Resting Bid

RISK	POOLS	RANGE	HOLD
Medium	X/SOL only	Below · skew 1	Waits for a dip

The idea. A pure limit-buy ladder that pays you fees while it fills. Restrict the agent to SOL-quoted pools and put the entire range below the price. The deposit is 100% SOL: no funding swap, and if the dip never comes, no exit swap either. When price does fall into the range, the position buys the base asset gradually and earns fees on every swap that moves it.

It suits tokens you would be happy to own a little cheaper, in pools with high turnover, during periods of chop.

It fails two ways. If price never dips, the position earns nothing. Because it starts out of range, the out-of-range timer is what decides how long it waits, so that timer must be lengthened or every position is closed after 25 minutes having done nothing. And if price falls straight through the range, you own the base asset at the average of the range, now worth less.

```
{
  "quoteMints": ["SOL"],
  "rangePreset": "ladder_bid",
  "rangeSkew": 1,
  "outOfRangeWaitMinutes": 240,
  "maxHoldHours": 48,
  "minVolumeTvlRatio": 3
}
```

Watch how often positions close out of range without ever having been in range. If most do, the ranges are too far below the price for the pools chosen. A slightly lower `adaptiveRangeWidthFactor` brings the top edge closer.

3 · The SOL Take-Profit Ladder

RISK	POOLS	RANGE	HOLD
Low-medium	SOL/USDC	Above skew 0	Waits for a rally

The idea. On a SOL/USDC pool, SOL is the base asset. A range placed entirely above SOL's price is held entirely in SOL, so the deposit needs no swap. If SOL rallies into the range, the position sells SOL for USDC gradually as the price climbs, earning fees on the way. It is a take-profit order book that pays you to rest.

This is a good place to use Aeternum as a hybrid: *you* choose the entry from the command line, and the exit engine manages it from there.

```
# check tick spacing, fee tier and TVL first
aeternum pool <SOL/USDC pool address>

# dry run; add --live only when it looks right
aeternum open --pool <address> --sol 1 --width 10 --skew 0 \
  --reason "selling SOL into strength above price"
```

```
{
  "outOfRangeWaitMinutes": 720,
  "maxHoldHours": 168,
  "autoSwapToSol": false
}
```

Two cautions. First, `autoSwapToSol` sells everything a closed position returns back to SOL, which would undo the very sale this strategy exists to make. Turn it off if you want to keep the USDC, and remember the setting is global. Second, like the Resting Bid, this range starts out of range and needs a long out-of-range wait, which then applies to every position the process holds. Run it with `maxPositions: 1`, or in its own instance.

4 • The Wide Harbour

RISK	POOLS	RANGE	HOLD
Lower	Deep pools	Wide	Days

The idea. Give up fee density in exchange for calm. Only deep pools of established tokens, wide symmetric ranges, long holds and tight stops. Positions rarely leave their range, divergence is gentle, and the agent trades rarely, so funding costs are paid less often. This is the conservative profile, slightly extended.

It suits operators who want to check in once a day, larger wallets where per-position fixed costs are negligible, and pairs with real, steady volume.

It fails quietly, by earning too little. A wide range on a deep pool can sit in range for a week and still lose to holding SOL. The low-yield exit is your friend here; do not disable it.

Choose the *conservative* profile in `npm run setup`, then add:

```
{
  "rangePreset": "wide",
  "adaptiveRangeWidth": true,
  "adaptiveRangeWidthFactor": 1.2,
  "minRangeWidthPct": 15,
  "maxPriceDelta24h": 0.25,
  "maxHoldHours": 168,
  "minFeeAprToHold": 0.2
}
```

5 • The Fee Sprinter

RISK	POOLS	RANGE	HOLD
High	High-turnover	Narrow	Minutes to a day

The idea. Chase the densest fees available: narrow ranges on pools that turn over their liquidity many times a day, quick exits the moment the range is left, short maximum holds. When it works, fee income per hour is

many times that of a wide range.

It suits experienced operators with larger wallets on a fast, paid RPC, who have watched it in paper mode long enough to trust the exits.

It fails on cost. Every exit and re-entry pays funding swaps and fixed costs again, and a sprinter exits often. On a small wallet, or on X/USDC pools where every leg is bought, those costs can exceed the fees. It also fails on gaps: a narrow range offers no cushion, so a sudden move is pure divergence.

Choose the *aggressive* profile in `npm run setup`, then add:

```
{
  "rangePreset": "tight",
  "minRangeWidthPct": 4,
  "adaptiveRangeWidthFactor": 0.6,
  "minVolumeTvlRatio": 4,
  "outOfRangeWaitMinutes": 10,
  "maxHoldHours": 24,
  "watcherIntervalSec": 15
}
```

Watch fees earned against funding cost paid, per position. If the second is regularly more than half the first, the sprinter is working for the swap venue, not for you.

6 • Small-Wallet Mode

RISK	WALLET	RANGE	HOLD
Medium	Under ~2 SOL	Adaptive • 0.95	Longer

The idea. On a small wallet the enemy is not divergence but overhead. Fixed costs are a larger share of every position, every swap fee bites, and churn is fatal. Small-wallet mode does everything it can to enter cheaply and stay longer: one position at a time, SOL-quoted pools only so there is

always a cheap shape, a skew that keeps almost the whole deposit in SOL, no creator fee, a higher bar on fee yield so the pools chosen can pay for their overhead, and exits that do not fire on noise.

```
# .env
AETERNUM_REFERRAL_FEE_BPS=0
```

```
{
  "maxPositions": 1,
  "quoteMints": ["SOL"],
  "rangeSkew": 0.95,
  "gasReserveSol": 0.1,
  "minDeploySol": 0.1,
  "deploySol": 0.2,
  "minFeeApr": 0.8,
  "minVolumeTvlRatio": 2,
  "trailingTriggerPct": 3,
  "trailingDropPct": 1,
  "outOfRangeWaitMinutes": 40
}
```

Set expectations honestly. Aeternum's own first live position was about \$28. In a normal pool that earned on the order of thirty cents a day in fees, about the pool's average rate for its size. That is the true scale of a small wallet: the strategy can be correct and the absolute numbers still tiny. Small-wallet mode is for learning the system with real money at risk, not for income.

Choosing between them

IF YOU...	START WITH
Are new, and want to see the system as designed	Core Bid, in paper mode
Have under two SOL	Small-Wallet Mode
Want to accumulate a token you like on dips	Resting Bid
Hold SOL and want to sell some into a rally	SOL Take-Profit Ladder
Want to check in once a day	Wide Harbour
Have size, a fast RPC and patience for tuning	Fee Sprinter

ONE STRATEGY PER PROCESS

Settings are global to a running agent, and exactly one process may run per ledger and bot token: two Telegram pollers on one token fight each other, and two watchers would double-submit closes. To run two strategies at once, use two separate copies of the project, each with its own directory, wallet and Telegram bot.

Exits are arithmetic

Six rules, checked in order every twenty seconds, with no model in the path. How each one works and how to tune it.

AN ENTRY CAN BE WRONG AND STILL TURN OUT FINE. AN EXIT THAT IS LATE, INCONSISTENT or skipped compounds every other mistake. That is why, in Aeternum, the function that closes positions, `evaluateExit`, is plain code. It runs on every watcher tick and every management cycle, it is deterministic, and it does not care what the model thinks.

The rules, in order

Rules are ordered by severity: capital preservation first, profit taking second, opportunity cost last. The first one that fires, and is confirmed, closes the position.

#	RULE	DEFAULT	FIRES WHEN
1	Stop loss	-12%	PnL is at or below the floor
2	Trailing take-profit	4% / 1.5%	Armed once peak PnL reaches the trigger; fires when PnL falls the drop distance from the peak
3	Take profit	8%	PnL reaches the hard ceiling
4	Out of range	25 min	Price has been outside the range for at least this long
5	Maximum hold	72h	The position is older than this, whatever its PnL
6	Low yield	15% APR	After a 90-minute grace period, fee APR has dropped below the floor

If none fires, and uncollected fees exceed `autoHarvestFeesUsd` (\$5), the fees are harvested to the wallet without closing.

Confirmation: why one bad read must not close a position

RPC nodes occasionally return a stale or wrong account. If the exit engine trusted every read, one inflated price would raise the peak, arm the trailing stop, and the next, *correct*, read would look like a sharp drop from the peak, closing a healthy position at a loss. Aeternum defends against this twice. A new peak is only accepted after `confirmTicks` consecutive reads (default 2) stay above the old one, and an exit signal must repeat on the same number of consecutive evaluations before it fires. At a 20-second watcher interval that adds about forty seconds, which is far cheaper than a phantom exit.

The trailing take-profit, step by step

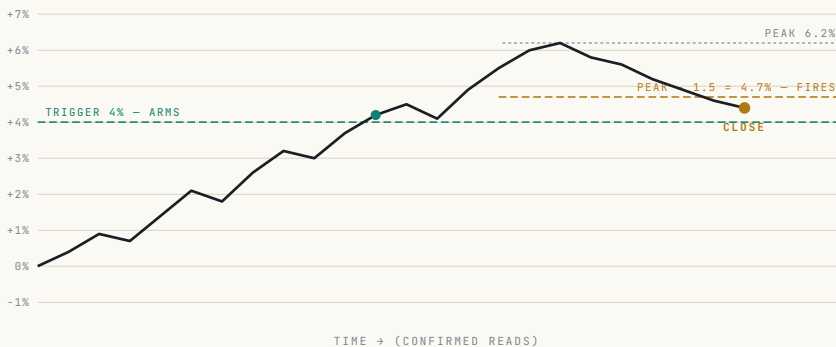


FIG. 7.1 Trailing take-profit with the default trigger and drop. Arming is sticky; the exit fires once PnL is confirmed 1.5 points below the highest confirmed peak.

With the defaults (trigger 4%, drop 1.5%): the position rises and its confirmed peak climbs. At a confirmed peak of 4% the trailing stop **arms**, and arming is sticky, so a later dip does not disarm it. The peak keeps rising to, say, 6.2%. From then on, the position closes the first time PnL is confirmed

at or below 4.7%, giving back no more than 1.5 points from the best it reached. If the position rallies to the 8% hard ceiling first, the take-profit rule closes it there instead.

How the rules interact with range width

For a narrow range the stop loss is rarely the rule that fires. In a 12% symmetric range, price at the bottom edge has cost the position only about 4.5%. To reach -12% it must fall well below the range, and the out-of-range timer will usually have closed it first. The stop loss is there for fast crashes and gaps, when price moves through the range and beyond between two reads. For wide ranges the reverse is true: price can lose a lot without leaving the range, and the stop loss does most of the protecting.

Tuning principles

- **The trigger must clear your costs.** If a round trip costs 1% and the trailing stop arms at 2% with a 1% drop, a trailing exit can bank +1% of position PnL and break even in the wallet. Keep `trailingTriggerPct - trailingDropPct` comfortably above your round-trip cost, or switch the exit basis to net PnL (Chapter 11).
- **The drop should be proportionate.** A drop wider than half the trigger gives back most of what it armed on.
- **A trigger set too high never arms.** If positions keep peaking at +2 or +3% and then closing at a loss, the trailing stop never got the chance to work. Aeternum's evolution logic looks for exactly this pattern.
- **The out-of-range wait is a bet on mean reversion.** Short waits cut losers fast but also cut positions that would have come back. Long waits suit one-sided ranges that start outside the price.
- **Maximum hold keeps capital honest.** A position that has done nothing for three days is usually better redeployed.

Pausing

`/pause` in Telegram or the terminal stops the exit rules; `/resume` restarts them. Pausing is for maintenance, such as moving servers or changing RPC. While paused, nothing protects your positions. Resume promptly.

After the close

A close withdraws liquidity, collects any remaining fees and burns the position NFT, refunding its rent. With `autoSwapToSol` on (the default), Aeternum then sells what the position returned back to SOL, and only what this position returned: it measures balances before the close, so it never sells tokens you already held. The closed record, with its exit reason, peak, time in range and PnL, is what feeds the learning described in Chapter 9.

Sizing and risk

How much goes into each position, the ceilings that cannot be argued with, and keeping the wallet itself safe.

Position size

Unless the model asks for a specific amount, each position is sized by a formula that compounds with the wallet but stays inside hard bounds:

```
deployable = wallet SOL - gasReserveSol (0.15)
size       = deployable × positionSizePct (0.35)
size       = at least max(minDeploySol, deploySol), at most
maxDeploySol
size       = never more than deployable
```

WALLET	DEPLOYABLE	35% OF IT	POSITION	WHY
0.6 SOL	0.45	0.16	0.45	Floor of 0.5, capped at deployable
2 SOL	1.85	0.65	0.65	Formula applies
10 SOL	9.85	3.45	3.45	Formula applies
100 SOL	99.85	34.9	25	Capped by <code>maxDeploySol</code>

THE SMALL-WALLET SURPRISE

Look at the first row. Because `deploySol` (default 0.5) acts as a floor, a small wallet puts almost everything into its first position. If that is not what you want, lower `deploySol` and `minDeploySol` together, as Small-Wallet Mode does.

The ceilings

CONTROL	DEFAULT	ENFORCED BY
maxPositions	3	Open refused at the limit
maxDeploySol	25	Size silently capped
minWalletSolToOpen	0.4	Open refused below it
gasReserveSol	0.15	Never deployed, so closes can always be paid for
onePositionPerToken	on	Two ranges on one asset is a doubled bet, not diversification
Re-entry cooldown	2 losses → 12h	A pool that lost twice in a row is skipped for twelve hours
Blocklist	—	Pools or mints you never want touched

None of these is in a prompt. The model cannot talk its way past them, and the evolution logic in the next chapter is forbidden from changing `maxPositions`, `maxDeploySol` or `stopLossPct`. Those are your decisions.

Keeping the wallet safe

An agent that signs transactions needs a hot key on an always-on machine. Treat that as the real risk it is.

- ☐ Use a **dedicated wallet** that holds only what the agent is meant to trade. Never your main wallet.
- ☐ Keep the key only in `.env`, with permissions `600`. Never in `user-config.json`, which is the file people paste into bug reports.
- ☐ **Never paste a private key** into a chat, an AI assistant, a screenshot or a support ticket. If you ever have, assume it is compromised: move the funds to a fresh wallet and use the new key.
- ☐ The same goes for the Telegram bot token. If it leaks, revoke it with `/revoke` in BotFather and issue a new one.

- ❑ Set `TELEGRAM_ALLOWED_USER_IDS` . The bot never adopts whoever messages it first, and in groups only listed users can issue commands.
- ❑ Keep the dashboard on loopback, or set `dashboardToken` before binding it to your network. It shows positions and PnL.
- ❑ Use a paid or private RPC for live trading. The public endpoint rate-limits the watcher, and a rate-limited watcher is a slow stop loss.

Learning: lessons, evolution, hivemind

Three ways the agent improves with experience, each bounded so that improving can never become drifting.

AETERNUM LEARNS IN TWO DELIBERATELY SEPARATE WAYS. **LESSONS** SHAPE JUDGEMENT: short written rules that go into the model's prompt. **Evolution** shapes mechanics: bounded arithmetic adjustments to numeric thresholds. Lessons can be wrong and are cheap to drop. Evolution changes what the agent is allowed to buy, so it is conservative, logged and reversible.

Lessons

A lesson is one sentence, such as "pools whose volume is mostly one wallet reverse within hours", tagged and stored in `data/lessons.json`. Lessons come from three places: the agent writes them when it notices something, you add them yourself, and Aeternum derives them automatically from closes that actually taught something (an unusual loss, a range left far too quickly). Ordinary, unremarkable exits produce nothing, because noise in the prompt costs accuracy. You can pin lessons you want kept, and remove ones that have gone stale. `/lessons` in Telegram shows what it currently believes.

Evolution

Evolution runs when you ask for it, with `/evolve` or `aeternum evolve`, and `aeternum evolve --dry-run` shows the proposals without applying them. It needs at least five closed positions, and every proposal carries the evidence that produced it.

EVIDENCE	PROPOSAL
$\geq 40\%$ of exits out of range	Widen: width factor $\times 1.25$
In range $> 90\%$ of the time, $\leq 10\%$ out-of-range exits, ≥ 8 closes	Tighten: width factor $\times 0.9$
Win rate below 40%	Min Yield Score $+5$; min fee APR $\times 1.15$
Win rate above 65% over ≥ 10 closes	Min Yield Score -3
$\geq 30\%$ of exits are stop losses	Max 24h move $\times 0.85$
$\geq 30\%$ of exits are dead yield	Shorten the yield grace period
Trailing exits average under 60% of the trigger	Arm later: trigger $\times 1.2$
Positions peak just under the trigger, then lose	Arm earlier, and keep the drop proportionate

Every value is clamped to a sane band, and the risk ceilings are never touched.

Pool memory

Every entry and exit is also remembered per pool. After two consecutive losing exits a pool goes on a twelve-hour cooldown, so the screener stops paying tuition on the same lesson. Both numbers are settings: `reentryCooldownLosses` and `reentryCooldownHours`.

Hivemind: learning together

One agent learns only from its own closes, which is a slow and expensive curriculum. The optional hivemind lets independent agents pool their conclusions. It is off until you give it a URL, and there is no default server; you host your own with `npm run hivemind:serve` , which listens only on the local machine unless you deliberately open it.

SHARED	NEVER SHARED
Derived lessons; closed outcomes as pool, pair, preset, width, PnL %, fee APR, hold time, time in range, exit reason	Private keys, wallet addresses, balances, position mints, transaction signatures, dollar amounts

Trust in the swarm is agreement, not authority. The server counts how many *distinct* agents independently reported each lesson and serves them in that order, never echoing a lesson back to its own author as consensus. Inbound lessons reach the model labelled as reports from other operators, explicitly not as instructions.

PART III

Operation

Setting it up, judging it honestly, and what really happened when it met mainnet.

From paper to live

Install, configure, run on a simulated balance for days, then fund it. In that order, every time.

What you need

- An always-on Linux machine or small VPS with **Node.js 20** or newer.
- A **Solana RPC** endpoint. The public one works for paper mode; use a paid or private endpoint for live trading, ideally with a second cheap one for the watcher.
- Any **OpenAI-compatible** `/chat/completions` endpoint that supports tool calling: OpenRouter, OpenAI, a gateway, or a local server such as LM Studio, Ollama or vLLM.
- Optionally, a **Telegram bot** from @BotFather.

Install

```
git clone https://github.com/pecundangcrypto/aeternum
cd aeternum
npm install
# two-minute wizard; choose a risk profile
npm run setup
```

The wizard writes two files, both ignored by git. `.env` holds secrets: wallet key, RPC URLs, API keys and `DRY_RUN`. `user-config.json` holds behaviour: thresholds, range geometry, exits and schedule. Keep that separation. Secrets never belong in the second file.

The model endpoint

```
# .env
# the API root, usually ending in /v1
LLM_BASE_URL=https://openrouter.ai/api/v1
LLM_API_KEY=...
```

```
{ "screenModel": "model-id-your-endpoint-uses",
  "manageModel": "model-id-your-endpoint-uses",
  "chatModel": "model-id-your-endpoint-uses" }
```

Test it before leaving it alone: `aeternum ask "which candidate looks best right now and why?"` should come back with an answer that clearly used the candidate data. Tool calling is mandatory, and an endpoint that silently ignores tools produces an agent that talks and never acts. Reasoning models need room to think: if responses come back empty, raise `maxTokens` (16000 is a sensible value for them).

Aeternum watches its own endpoint. If three consecutive cycles run but inspect nothing, because the key is out of credit or the model is failing, it pauses screening, alerts you once, and re-probes every thirty minutes instead of burning cycles silently.

Paper mode: the first week

Leave `DRY_RUN=true` and **do not** put a wallet key in `.env`. The agent then runs on a synthetic account (10 SOL by default, `paperStartingSol`) and is structurally unable to sign, because there is nothing to sign with.

```
# model, Orca, Jupiter, hivemind, and no key present
scripts/preflight.sh

# systemd units: survive reboot, restart on crash
scripts/paper-run.sh install
scripts/paper-run.sh logs
```

Paper positions are valued against *live* pool state with the same functions the on-chain program uses, so value, divergence and range status are exact. Only fee income is estimated, from the pool's trailing fee flow and the share of active liquidity the position would represent. Every paper snapshot is flagged `estimated: true`. The exit engine treats paper positions exactly like real ones, so stops, trailing take-profit and out-of-range timers exercise the same code paths they will live.

WHAT TO LOOK AT DURING THE PAPER WEEK

- `aeternum candidates --limit 5`: is the screener finding pools you would consider reasonable?
- `aeternum decisions --kind close`: do the exit reasons make sense?
- `aeternum performance`: win rate, exit mix, time in range, average hold.
- The paper result *against holding SOL* over the same days. Beating zero is not the bar.

Telegram

```
TELEGRAM_BOT_TOKEN=123456:ABC...
# message your bot first, then:
node scripts/telegram-link.js
# saves the chat id and allowed user id
node scripts/telegram-link.js --write
```

The bot gives you a persistent keyboard (Status, Positions, Screen, Manage, Candidates, Performance, Config, Lessons, Pause, Resume). `/positions` draws each position with a range bar and marks an armed trailing stop. Each position has a close button that asks for confirmation and names the exact position before acting. Closes are always addressed by list index, never by a typed address, so a typo cannot hit the wrong position. `/config` offers buttons to toggle trailing take-profit and nudge the take-profit, stop, trigger and drop live.

Going live

1. Create a **new, dedicated wallet** and fund it with a small amount.
2. Put its key in `.env`, set `DRY_RUN=false`, and `chmod 600 .env`.
3. Lower `maxPositions` to 1 for the first days.
4. Decide about the creator fee (`AETERNUM_REFERRAL_FEE_BPS`, Chapter 2).
5. Run under a supervisor, either `systemd` or `npm run pm2:start`, never a bare background process: a reboot will silently stop it, and with it every stop loss.
6. Watch the first open and the first close end to end, in Telegram and on the explorer.

Write commands on the CLI are dry runs unless you add `--live`. That flag is the only thing that signs. Always run a command once without it.

The dashboard

Set `dashboardEnabled: true` for a read-only status page on port 8788. It serves what the ledger already knows, so opening it costs no RPC calls. It binds to the local machine by default; to view it from your phone on the same network, set `dashboardHost` to `0.0.0.0` and a `dashboardToken`, then open `http://<machine-ip>:8788/?token=<token>`.

Measuring results honestly

Two kinds of PnL, why they disagree, and the handful of numbers that tell you whether the strategy is really working.

Position PnL

Position PnL answers the question *how is the liquidity doing?* It is measured entirely in the pool's quote asset, at the pool's own price, with no outside oracle:

```
position PnL = (current value + uncollected fees + already har-  
vested - entry value) ÷ entry value
```

Two design choices matter. Using one price source avoids injecting the basis between sources into the result: on one ZEC/USDC position the pool and an oracle disagreed by 0.17%, which showed up as a loss that never happened. And including already-harvested fees means harvesting never looks like a loss. Without that term, every harvest drops PnL, which can trip the trailing stop. Position PnL is also what Orca's own position page shows, so it is the number you can verify independently.

Net PnL

Net PnL answers a different question: *if I closed now and sold everything back to SOL, how much would the wallet be up or down?* It starts from the SOL that actually left the wallet to open the position, adds rent that will come back, and values every token the close would return (position, fees and

entry leftovers) at market prices less the expected cost of selling it. It is only available for live positions, because only they have a measured entry cost.

Why they disagree, and which one to use

The gap between the two is mostly funding cost. On a cheap, zero-swap shape they are close. On an X/USDC position with both legs bought, net PnL starts around a percent or more below position PnL and stays there.

The `pnlBasis` setting chooses which drives the exits and leads the displays. `"position"`, the default, matches Orca's page and is easy to check. `"net"` is stricter: a trailing stop judged on net PnL cannot "take profit" at a level that is a loss in the wallet. Net PnL is recorded for every live position either way. If you keep the default, compensate by keeping your trailing trigger and drop well clear of your round-trip cost (Chapter 7).

A LESSON IN DISPLAY DESIGN

For a while, net PnL was the headline number on every screen. In practice it confused more than it helped: it diverged from Orca's page, and there was no way to check it independently. The headline went back to position PnL, with net PnL kept alongside. Show the number people can verify, and keep the stricter one one tap away.

Paper results are estimates

Paper fee income is modelled, not earned, and every paper snapshot says so. Treat a paper week as a test of the *behaviour* (does it pick sensible pools, size sensible ranges, exit for sensible reasons?) more than of the precise return.

The numbers that matter

METRIC	WHAT IT TELLS YOU
Return vs holding SOL	Whether liquidity provision beat doing nothing. The only bar that counts
Exit mix	Which rule closes most positions, and the win rate of each
Time in range	Whether ranges are sized right. Low means too narrow; very high with few out-of-range exits means too wide
Fees vs funding cost	Whether the pools chosen can pay for getting in and out
Average hold	Churn. Short holds multiply fixed costs
Results by preset and pair	Where the edge, if any, actually comes from

Sample size

Ten closed positions tell you very little. Liquidity provision has a lopsided distribution of outcomes: many small wins from fees, occasional large losses from divergence. A week of winners can be erased by one gap down. Judge a configuration over dozens of closes, across different market conditions, and be suspicious of any conclusion drawn from a single good or bad week, including your own.

Field notes from a live deployment

What broke, what it cost, and what changed because of it. Every fix described here is in the code today.

NO AMOUNT OF DESIGN SURVIVES CONTACT WITH MAINNET UNCHANGED. THESE ARE the problems found while building and running Aeternum, in paper mode and with real money, including the ones that cost money. They are here because each one carries a lesson that applies to any automated liquidity system, not just this one.

1• The open that funded, then failed

The first live open bought both legs of a DOGE/USDC position from SOL, then failed to build the position transaction with "payer not set". The Orca SDK's action functions use a module-level payer that must be registered separately from the wallet the agent was signing with. Paper mode never signs, so it never exercised that path. The result was a wallet holding DOGE and USDC it had not meant to buy, swapped back by hand.

What changed: the payer is registered and checked against the wallet at startup, and if an open fails after funding but before anything reaches the chain, the funding swaps are automatically unwound. **The lesson:** paper mode tests strategy, not signing. The first live run of any write path should be the smallest amount possible, watched end to end.

2 • The 1.7% round trip

Once that position was open, its PnL on Orca's page looked fine, while the wallet told a different story. Measuring before and after showed the round trip (buying both legs, and the swaps needed to sell them back) cost about 1.7%, roughly the whole edge of a typical position.

What changed: entry costs are measured from the wallet for every live position, net PnL is computed from them, and the screener ranks pools by funding cost as well as yield. That is the whole of Chapter 2.

3 • A loss that never happened

Early PnL mixed price sources: an oracle for one leg, the pool price for another. On a ZEC/USDC position they disagreed by 0.17%, which appeared as 0.08% of loss. Across many small exits, a bias that size decides the win rate.

What changed: all PnL is in the pool's quote asset at the pool's own price. USD is a display conversion applied at the very end and never fed back.

4 • One bad read

This one was designed against from the first day, because it is the classic failure of any automated trailing stop. A single anomalous RPC read inflates a position's peak. That arms the trailing stop, and the next, correct, read looks like a sharp drop from the peak, closing a healthy position at a loss.

What changed: peaks and exit signals both require consecutive confirmation (`confirmTicks`). Never set it to 1 on a live wallet.

5 • The deposit that was a hundred times too big

The SDK reports a range's deposit split in basis points that sum to 10,000. Treated as percentages, every deposit would have been sized a hundred times too large. It was caught before any live deposit, because in a dry run the numbers were plainly absurd.

What changed: the conversion lives in one function, `depositSplit`, rather than at every call site. **The lesson:** units are where money disappears. Paper mode earns its keep by making unit errors visible before they are expensive.

6 • A hundred and forty-six silent cycles

The model endpoint ran out of quota. Every screening cycle "ran", inspected nothing, and logged no error worth noticing. Nothing was opened for days, and nothing said why.

What changed: endpoint health tracking. Three empty cycles in a row pause screening, send one alert, and start a thirty-minute re-probe.

7 • The reboot

An early multi-day run was started as a plain background process. The machine rebooted; the agent did not come back, and with it went every exit rule.

What changed: systemd units and a pm2 configuration ship with the project. Anything holding open positions must run under a supervisor.

8 • Two books in one ledger

A dry-run process could see real positions in the ledger and, in principle, act on them. Nothing went wrong, but nothing structural prevented it.

What changed: paper and live positions are separate books. Close, harvest, reduce and sweep all refuse a position from the other book, with a message saying so.

9 • The empty answer

A reasoning model spent its whole token budget thinking and returned no text and no tool call. The cycle looked like a decision to do nothing.

What changed: a larger token budget for reasoning models, a nudge when a turn comes back empty, and logging that distinguishes "decided nothing" from "said nothing".

10 • The scale of a small wallet

The first live position was about \$28. In range, in a normal pool, earning about the pool's average rate for its size, it made on the order of thirty cents a day. The system worked as designed, and the absolute numbers were still tiny.

The lesson: fee income scales with capital; overhead mostly does not. Small wallets are for learning. Size is what turns a correct strategy into a meaningful one, and size is exactly what you should not add until a strategy has proven itself.

Checklists and troubleshooting

The pages to keep open while you run it.

Before going live

- ☐ Paper mode ran for several days, under a supervisor, without interruption.
- ☐ You have read the exit reasons of every paper close and they make sense.
- ☐ The paper result has been compared with holding SOL over the same period.
- ☐ A dedicated wallet, funded with an amount you can lose entirely.
- ☐ `.env` is `chmod 600` and the key has never been pasted anywhere else.
- ☐ A paid or private RPC, ideally with a separate watcher RPC.
- ☐ `maxPositions` is 1 for the first days.
- ☐ Telegram works, with `TELEGRAM_ALLOWED_USER_IDS` set.
- ☐ You know how to `/pause`, how to close a position by index, and how to stop the service.
- ☐ You have decided about the creator fee.

Daily, in two minutes

- ☐ `/status`: the agent is running, the watcher is alive, the model endpoint is healthy.

- ☐ `/positions`: every position in range, or with a sensible reason not to be.
- ☐ Any unexpected Telegram alerts read and understood.

Weekly

- ☐ `aeternum performance`: exit mix, time in range, average hold.
- ☐ `aeternum evolve --dry-run`: read the proposals and the evidence, then decide.
- ☐ Prune stale lessons; pin the ones that keep proving right.
- ☐ Compare against holding SOL. Be honest about the answer.

Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
Nothing ever opens	Read the rejection reasons from <code>aeternum candidates</code> . Usually thresholds too strict for the day's market, a wallet below <code>minWalletSolToOpen</code> , or an unhealthy model endpoint
Telegram HTTP 409	Two processes are polling the same bot token. Run exactly one
Agent replies but never acts	The endpoint or model does not really support tool calling. Test with <code>aeternum ask</code> ; switch model
Empty model responses	Reasoning model out of token budget. Raise <code>maxTokens</code>
RPC 429 errors, slow exits	Public or overloaded RPC. Use a paid endpoint and a separate <code>WATCHER_RPC_URL</code>
PnL slightly differs from Orca's page	Small timing and rounding differences are normal. Large ones usually mean you are reading net PnL
"Refused: ..." from a command	A gate blocked it, and the message says which. Working as intended
Stray tokens after a failed open	Should be unwound automatically. If any remain, swap them back to SOL with the CLI (dry run first)
Positions out of range too often	Raise <code>adaptiveRangeWidthFactor</code> , or lower <code>maxPriceDelta24h</code>
Agent stopped after a reboot	Not running under systemd or pm2. Install the service

Settings reference

The settings this book discusses, with their shipped defaults. All live in `user-config.json` unless marked `.env`.

CAPITAL AND RISK

KEY	DEFAULT	MEANING
<code>maxPositions</code>	3	Concurrent positions
<code>maxDeploySol</code>	25	Largest single position
<code>minDeploySol</code>	0.2	Smallest position worth opening
<code>deploySol</code>	0.5	Size floor for the automatic formula
<code>positionSizePct</code>	0.35	Share of deployable balance per position
<code>gasReserveSol</code>	0.15	Never deployed
<code>minWalletSolToOpen</code>	0.4	No opens below this wallet balance
<code>onePositionPerToken</code>	true	One position per base asset
<code>reentryCooldownLosses</code> / Hours	2 / 12	Pool cooldown after consecutive losses

SCREENING

KEY	DEFAULT	MEANING
quoteMints	SOL, USDC	Assets PnL may be measured in
minTvlUsd / maxTvlUsd	25k / 3M	Pool depth band
minVolume24hUsd	50k	Minimum daily volume
minFeeApr	0.4	Fee APR floor (0.4 = 40%)
minVolumeTvlRatio	1.5	Daily turnover floor
maxPriceDelta24h	0.6	Volatility gate (0.6 = 60%)
min/maxTickSpacing	1 / 256	Allowed grids
maxTokenRisk	4	Orca's 0–10 token risk
min/maxMcapUsd	250k / 500M	Market-cap band
minHolders	400	Holder floor
maxTop10Pct	60	Top-10 concentration ceiling
minTokenAgeHours	24	Token age floor
minYieldScore	45	Score floor, 0–100
candidateLimit	8	Candidates shown to the model
signalMode	merge	merge or only

RANGE

KEY	DEFAULT	MEANING
rangePreset	ladder_bid	One of the five presets
rangeWidthPct / rangeSkew	null	Global overrides
adaptiveRangeWidth	true	Size width from volatility
adaptiveRangeWidthFactor	0.75	Multiplier on the 24h move
adaptiveTurnoverFactor	3	Multiplier on $\sqrt{\text{turnover}}$
min/maxRangeWidthPct	6 / 60	Width clamp (restart to apply)

EXITS

KEY	DEFAULT	MEANING
<code>stopLossPct</code>	-12	Hard floor
<code>trailingTakeProfit</code>	true	Enable trailing
<code>trailingTriggerPct</code>	4	Peak that arms it
<code>trailingDropPct</code>	1.5	Give-back allowed from peak
<code>takeProfitPct</code>	8	Hard ceiling
<code>outOfRangeWaitMinutes</code>	25	Patience outside the range
<code>maxHoldHours</code>	72	Stale-capital limit
<code>minFeeAprToHold</code>	0.15	Dead-yield floor
<code>yieldGraceMinutes</code>	90	Before dead-yield applies
<code>confirmTicks</code>	2	Consecutive reads to confirm
<code>autoHarvestFeesUsd</code>	5	Harvest threshold
<code>autoSwapToSol</code>	true	Sell close proceeds to SOL
<code>pnlBasis</code>	position	position or net

SCHEDULE, MODEL AND ENVIRONMENT

KEY	DEFAULT	MEANING
<code>screenIntervalMin</code>	30	Screening cycle
<code>manageIntervalMin</code>	10	Management cycle
<code>watcherIntervalSec</code>	20	Exit-engine tick
<code>screen/manage/chatModel</code>	—	Model ids your endpoint uses
<code>maxTokens</code>	4096	Raise for reasoning models
<code>DRY_RUN (.env)</code>	true	No signing; paper mode without a key
<code>LLM_BASE_URL , LLM_API_KEY (.env)</code>	—	Model endpoint
<code>RPC_URL , WATCHER_RPC_URL (.env)</code>	—	Main and watcher RPC
<code>AETERNUM_REFERRAL_FEE_BPS (.env)</code>	50	Creator fee; 0 disables

Glossary

Base / quote

The asset being farmed, usually the volatile one, and the asset value is measured in: SOL, USDC or USDT.

Concentrated liquidity

Liquidity provided only within a chosen price range, rather than across all prices.

Divergence (impermanent) loss

The shortfall of a liquidity position against simply holding its assets, caused by the pool rebalancing against you as price moves.

Fee APR

A pool's last 24 hours of fees divided by its TVL, annualised. A description of the recent past, not a forecast.

Funding cost

The swap fees and spread paid buying a position's non-SOL assets on entry and selling them on exit.

Harvest

Collecting a position's accrued fees to the wallet without changing its liquidity.

Hivemind

An optional, self-hosted server through which independent Aeternum agents share anonymous lessons and outcomes.

Net PnL

What the wallet would gain or lose, in SOL, if the position were closed now and everything sold back to SOL.

Paper mode

Running on a simulated balance with no private key on the machine; positions valued against live pool state.

Position PnL

The position's return in its quote asset at the pool price, including uncollected and harvested fees. Matches Orca's page.

Range efficiency

The share of observed time the price spent inside a position's range.

Skew

The share of a range's width placed below the base asset's price: 1 is fully below, 0 fully above, 0.5 centred.

Tick / tick spacing

The integer price grid of a Whirlpool (0.01% steps) and the interval at which range edges may be placed.

Trailing take-profit

An exit that arms once PnL reaches a trigger and fires when PnL falls a set distance from its peak.

Turnover

24-hour volume divided by TVL: how many times a pool's liquidity changed hands in a day.

TVL

Total value locked: the value of all liquidity in a pool.

Yield Score

Aeternum's 0–100 ranking of a pool from fee yield, turnover, depth and stability.

Zero-swap skew

The one-sided range shape that can be funded entirely from SOL: skew 1 when SOL is the quote, 0 when SOL is the base.

A E T E R N U M

The model chooses entries. Arithmetic chooses exits.

FREE & OPEN-SOURCE · MIT

github.com/pecundangcrypto/aeternum

PROJECT INSPIRED BY MERIDIAN

NOT FINANCIAL ADVICE. PROVIDING LIQUIDITY CAN LOSE MONEY.